VC4

版本控制與上線

VC4 — Version Control & Deployment

做出來只是一半。
改壞了要回得去，做好了要別人打得開——這兩件事這週一起解決。

Vibe Coding 專班 · 第 4 週 · 兩軌自本週起合流 | 謝楠楨

這一週結束時，你手上會多出三樣東西

1 一個公開的網址

- 別人用手機點進去就能用
- 不必安裝任何東西、不必你在旁邊
- 這是期末專題五項交付物之一
- 也是你第一次真的「做出一個東西」
- 之後每次改完推上去，網址自動更新

2 一個 GitHub 儲存庫

- 完整的修改歷史，每一次改了什麼都看得到
- 換一台電腦也拿得到你的專案
- 跨工具的通用交付格式
- 綁在單一平台的成果無法交接
- 至少 5 個訊息寫得清楚的 commit

3 一次 revert 的紀錄

- 本週的重點練習：故意讓 AI 改壞，再退回去
- 沒做過這一步的人，不會相信版本控制有用
- 改壞前後的畫面、用了什麼指令、結果
- 從此你敢讓 AI 做大幅度的修改
- 因為你知道回得去

第 3 項是本週真正的主角。 VC2 講過「越改越糟時該退回上一版」，但當時只能用複製資料夾的土法。這一週學正規做法——而且要親手做一次，不是看過就算。

本週地圖：哪幾頁要讀、哪幾頁要動手

頁	段落	怎麼用	預估時間
4–12	PART 01　Git 的心智模型	讀。先建立直覺，指令才記得住	25 分
13–17	PART 02　GitHub 是什麼	讀。含 .gitignore 與公開／私有的選擇	15 分
18–24	PART 03　部署	讀。三個平台、選型決策樹、靜態 vs 後端	18 分
25–43	PART 04　動手做（Step 1–6）	打開你的 v0.3 專案，跟著做。這是本週的主體	100–140 分
44–45	卡住了？照症狀查	不必從頭讀，卡住時再翻	—
46	常見的四個誤解	做完之後回頭看一次	5 分
47–48	指令速查卡	印出來貼在螢幕旁邊	—
49–51	自我檢核、本週交付與回顧	交作業前一定要看	10 分

兩軌自本週起完全合流。A 軌若使用的瀏覽器工具內建版本歷史，仍須另外做一次 GitHub 推送——GitHub 是跨工具的通用交付格式，綁在單一平台的成果無法交接給別人。

PART 01

Git 的心智模型

How Git Actually Thinks

指令只有幾個，記不住是因為不知道它在做什麼。

先建立一個直覺： commit 不是「存檔」。

為什麼「另存新檔」不夠用



VC2 教你「大改之前先複製一份備份」，那是土法。這一週學正規做法——差別不只在方便，更在於「你知道每一次改了什麼、為什麼改」，那是備份資料夾給不了的資訊。

commit 不是存檔，是拍一張可以回去的快照

一連串 commit 長什麼樣



每一個字母是一次 commit，它記錄了：

- 當下所有檔案的完整狀態（快照）
- 跟上一次比，哪幾行變了
- 你寫的一句說明（訊息）
- 時間與作者

所以你隨時可以「跳回 B 看看當時長什麼樣」

1 不是覆蓋，是累積

存檔會蓋掉舊的，commit 不會
A、B、C、D 全部都還在，隨時回得去

2 每一個點都要能跑

只在「東西還能動」的時候 commit
壞掉的狀態存起來，回去也沒用

3 訊息是給未來的你看的

「修正第 8 級上限」三個月後看得懂
「update」「fix」等於沒寫

4 本機就完成了

commit 只存在你的電腦裡，還沒上傳
要推上 GitHub 是另一個動作（push）

建立這個直覺之後，後面的指令就好懂了：commit 是「拍一張」、push 是「上傳」、revert 是「再拍一張把上一張的修改抵銷掉」、reset 是「把『現在在這裡』的箭頭往回搬」。

一次 commit 會經過三個地方

1

① 工作區

你
正
在
改

2

② 暫存區

g
i
t

3

③ 本機儲存庫

g
i
t

4

④ GitHub

g
i
t

為什麼中間要有「暫存區」：它讓你可以只挑一部分檔案來記錄。實務上你多半用 git add . （全部都挑），所以這一層對初學者幾乎是透明的——知道有它就好，不必深究。

commit 的節奏：什麼時候該存一個點

原則：每一個「還能跑」的狀態，立刻 commit

- 不是每天一次，也不是做完才一次——是「剛剛那個改動成功了」就存
- 判斷標準很簡單：現在打開來還能用嗎？能，就 commit
- 存得越密，出事時要回退的範圍越小

在這門課裡，這些時機一定要 commit

- ① 每修好一個 bug　② 每加完一個功能　③ 要 AI 做大幅修改之前
- 第 ③ 個最重要——它就是你的退路。沒有它， revert 也沒地方可退
- 本週的 Step 4 會實際示範這個順序：先 commit → 讓 AI 改壞 → revert 回來

常見的錯誤節奏

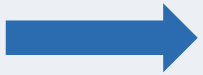
- 「等全部做完再一次 commit」——出事時只能整份重來
- 「壞掉的狀態也 commit」——回去也是壞的，等於沒存
- 「一天一次」——粒度太粗。以「一個可運行的改動」為單位，不是以時間

一個實用的自我檢查：如果現在電腦突然壞掉，你最多會損失多少工作？超過一小時，代表你 commit 得太少了。

commit 訊息怎麼寫

看不出改了什麼

不好的訊息
update
fix bug
修改
asdf
改了一些東西



一眼看得出改了什麼

好的訊息
初版：可以算出給付金額
修正低收自負額 0.05 → 0
修正第 8 級上限被壓回第 7 級
未選身分別時顯示提示訊息
加上 test.html，29 項測試

一句話的標準　三個月後的你，光看這一句就知道「要不要回到這一版」

寫什麼　改了什麼 + 為什麼（如果不明顯）。不必寫「怎麼改的」——程式碼本身會說

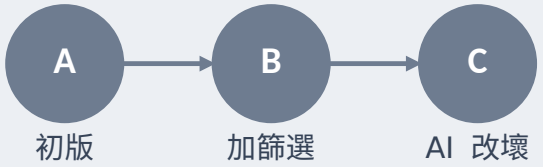
中文可以嗎　完全可以。這門課建議用中文，寫得清楚比寫得專業重要

助教會看這個　本週驗收條件之一：commit 訊息不得全是 update ／ fix

這件事看起來瑣碎，但它是「可交接」的基礎。一個 commit 訊息全是 update 的儲存庫，等於只有你自己看得懂——而期末專題的評分項目裡，「可交接」佔 8%。

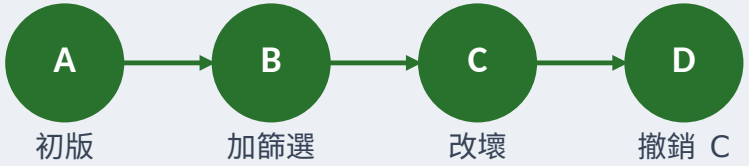
AI 把東西改壞了，三種退路差在哪

現況　還沒處理



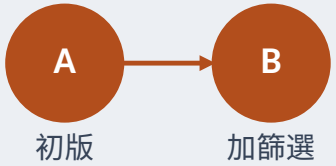
你現在在 C，東西壞了

git revert



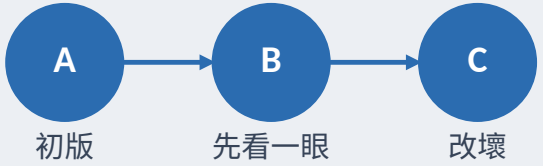
新增一個「做反向修改」的 commit。歷史完整保留，最安全，多人協作一定用這個

git reset



把指標直接搬回 B， C 從紀錄上消失。乾淨，但已推上 GitHub 就不要用

git checkout



只是「跳過去看看 B 長什麼樣」，什麼都沒改。確認完再跳回來

選哪一個：東西已經推上 GitHub → 一律用 revert（reset 會讓別人的紀錄對不上）。還在自己電腦裡、想把剛才的實驗整段丟掉 → reset 最乾淨。只是想知道「上週那版長什麼樣」→ checkout 看完再跳回來，什麼都不會動到。

branch 與 merge conflict：這一週不用做，但要知道

概念	白話的意思	什麼時候你會需要它
branch **分支**	從現在這一點岔出一條路， 在上面亂改都不影響原本那條	想試一個大改動，又不想毀掉現在能跑的版本時。 試成功就合併回來，試失敗就整條丟掉
merge **合併**	把岔出去那條路的改動， 併回主要那條	分支上的實驗成功了。 多數情況 Git 會自動處理好
merge conflict **合併衝突**	兩條路都改了同一行， Git 不知道該聽誰的	多人協作、或你在兩台電腦上都改了同一個檔案。 一個人做課堂專案幾乎不會遇到
衝突長什麼樣	檔案裡會出現 `<<<<<<<` 、 `=======` 、 `>>>>>>>` 三行標記	標記中間就是兩個版本。整段貼給 AI 問 「這兩個版本差在哪？我該保留哪一個？」

> 這一頁列在這裡是為了「遇到時知道那是什麼」，不是要你這週做。課堂專案一個人做，主線一條走到底就夠了——等到有第二個人一起改，或你開始做比較大的實驗時，再回來看這一頁。

兩個永遠安全的指令： status 與 log

在專案資料夾裡輸入 git status

```
$ git status

On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  modified:   calculator.js

Untracked files:
  test.html


$ git log --oneline

a1b2c3d 加上 test.html， 29 項測試
e4f5g6h 未選身分別時顯示提示訊息
i7j8k9l 修正第 8 級上限被壓回第 7 級
m0n1o2p 初版：可以算出給付金額
```

1 On branch main

你現在在主線上。這門課從頭到尾都在 main
不必開分支

2 modified： 改過但還沒記錄

calculator.js 被你（或 AI）改過了
還沒 add、還沒 commit

3 Untracked： 全新的檔案

test.html 是新的， Git 還不認識它
git add . 之後兩者都會被納入

4 log 的每一行 = 一個退路

左邊那串是編號， revert 時要用
右邊是你寫的訊息——寫得好，這裡就看得懂

這兩個指令永遠是安全的，因為它們只顯示、不修改任何東西。 Git 出狀況時的標準動作：先跑這兩個，把輸出整段貼給 AI 問「我現在在什麼狀態？該怎麼辦？」——比自己亂試指令安全得多。

PART 02

GitHub 是什麼

Git vs GitHub

Git 是你電腦裡的版本控制工具， GitHub 是放在網路上的儲存空間。
兩個是不同的東西，很多人一開始會搞混。

Git 與 GitHub：差在哪

比較項目	Git （本機）	GitHub （雲端）
它是什麼	你電腦上的一個工具（軟體）	一個網站（服務），把 Git 儲存庫放到雲端
要不要網路	不用。 commit 在飛機上也做得到	要。 push 與 pull 都需要連線
資料在哪	你的硬碟裡	GitHub 的伺服器上（也就是別人的電腦）
沒有它會怎樣	沒有版本歷史，改壞了回不去	電腦壞掉專案就沒了；別人也拿不到
這門課要用哪些指令	add 、 commit 、 revert 、 log	push 、 clone （ remote add 只做一次）
同類型的服務	—	GitLab 、 Bitbucket 。用法幾乎一樣，這門課用 GitHub
為什麼兩個都要	本機才有完整歷史，操作快	備份、交接、以及部署平台要從這裡抓程式碼

最後一列是本週的關鍵：部署平台（ Vercel 、 Netlify ）是「連到你的 GitHub 儲存庫，自動抓程式碼去上線」。所以沒有 GitHub ，就沒有那個一鍵更新的網址。

Public 還是 Private？醫療場域的判斷

建立儲存庫時要選公開或私有

1. 裡面有沒有任何真實個案資料？
   - 是 → 有 → 停。不論公開私有都不該推上去
     先把資料移除，改用模擬資料或公開資料
     本課程規範：一律不得使用真實個案資料
   - 否 ↓
2. 有沒有金鑰、密碼、內部網址？
   - 是 → 有 → 先設好 .gitignore 再推（下一頁）
     已經推上去的話，第一件事是到服務商後台撤銷那把金鑰
     刪 commit 沒有用——歷史與鏡像早就存下來了
   - 否 ↓
3. 這是要給別人看、或要放進履歷的嗎？
   - 是 → 是 → 選 Public 。課堂專案建議選這個
     公開才方便交作業、同儕互審、以及未來展示
     部署平台的免費方案通常也對公開儲存庫較友善
   - 否 ↓
4. 只是自己練習、還沒想好
   - → 選 Private 。之後隨時可以改成 Public
     但要注意：私有儲存庫的協作人數與部署額度可能受限
     兩者都可以，重點是先確認前兩題都是「沒有」

前兩題是紅線，第三、四題只是偏好。順序不能顛倒——很多人先選了 Public 才想到裡面有金鑰，那時候已經來不及了。

.gitignore：什麼不該推上去

不該推的東西	為什麼	寫在 .gitignore 裡
環境變數檔	裡面是金鑰與密碼。推上去等於公開， 幾分鐘內就會被自動掃走	.env .env.local
相依套件資料夾	動輒上萬個檔案，而且可以用一個指令重裝。 推上去只會讓儲存庫又肥又慢	node_modules/
作業系統產生的檔案	Mac 的 .DS_Store、Windows 的 Thumbs.db。 跟你的專案無關，只會製造雜訊	.DS_Store Thumbs.db
編輯器設定	那是你個人的偏好，不是專案的一部分	.vscode/ .idea/
建置輸出	從原始碼產生出來的東西，可以重新產生	dist/ build/
任何含真實資料的檔案	本課程紅線。個案清單、病歷、名冊一律不推	建議連放都不要放進專案資料夾

> .gitignore 要在「第一次 commit 之前」就設好。已經 commit 過的檔案，即使之後加進 .gitignore 也還留在歷史裡——這是最常見的金鑰外流原因。範例 \v1.0 附了一份可以直接抄的 .gitignore。

一次 push 到底發生了什麼事

1

① git add .

把改過的

2

② git commit -m

在本機拍

3

③ git push

把本機的

4

④ GitHub 收到

歷史紀錄

5

⑤ 部署平台

偵測到更

第 ⑤ 步是設定過一次之後就自動發生的——這也是為什麼值得花時間把部署接到 GitHub：之後你每改完一次，只要推上去，網址上的內容就自動更新，不必再手動上傳任何東西。

PART 03

部署：讓別人打得開

Deployment

「部署」聽起來很專業，其實只是把檔案放到一台永遠開著的電腦上。
對這門課的專案來說，它是免費的，而且大約五分鐘。

「部署」到底是什麼

	× 還沒部署	○ 部署之後
現在的狀態	檔案在你的硬碟 → 只有你打得開 → 要給別人看只能傳檔案 → 對方還要自己解壓縮	檔案在GitHub → 部署平台自動抓下來 → 放到一台永遠開的機器 → 給一個網址就結束了
要更新內容時	改完再傳一次 → 對方要重新下載 → 誰手上是哪版沒人知道 → 混亂	改完git push → 平台自動重新部署 → 網址不變，內容自動更新 → 所有人都是最新的
手機打得開嗎	要先把檔案傳到手機 → 多數手機打不開 html → 幾乎不可能 → 放棄	直接點網址 → 任何裝置都打得開 → 不必安裝任何東西 → 傳給誰都行

對這門課的專案（純前端、沒有資料庫）來說，部署是完全免費的，而且設定一次之後就不必再管。真正要花錢的是「需要後端與資料庫」的專案——那是第 5 週的話題。

四個部署平台：由簡到繁，本課程選最左邊那個

GitHub Pages

什麼時候選它

最單純的靜態網頁
本課程專案完全夠用

特色

GitHub 內建，
不必註冊第三方服務

注意

只支援靜態內容；
設定選項較少

Netlify

什麼時候選它

純靜態網站；想要
簡單的表單收集功能

特色

老牌、穩定，表單與簡單
後端函式的支援不錯

注意

建置設定（publish
directory）填錯會出空白頁

Vercel

什麼時候選它

用了 Next.js，或想要
最省事的自動部署

特色

前端框架支援最順，尤其
Next.js（同一家公司做的）

注意

免費方案常限非商業用途，
開課前請確認當期條款

Firebase Hosting

什麼時候選它

需要存資料或使用者登入
（第 5 週會提到）

特色

Google 的服務，同一個
平台就有資料庫與登入

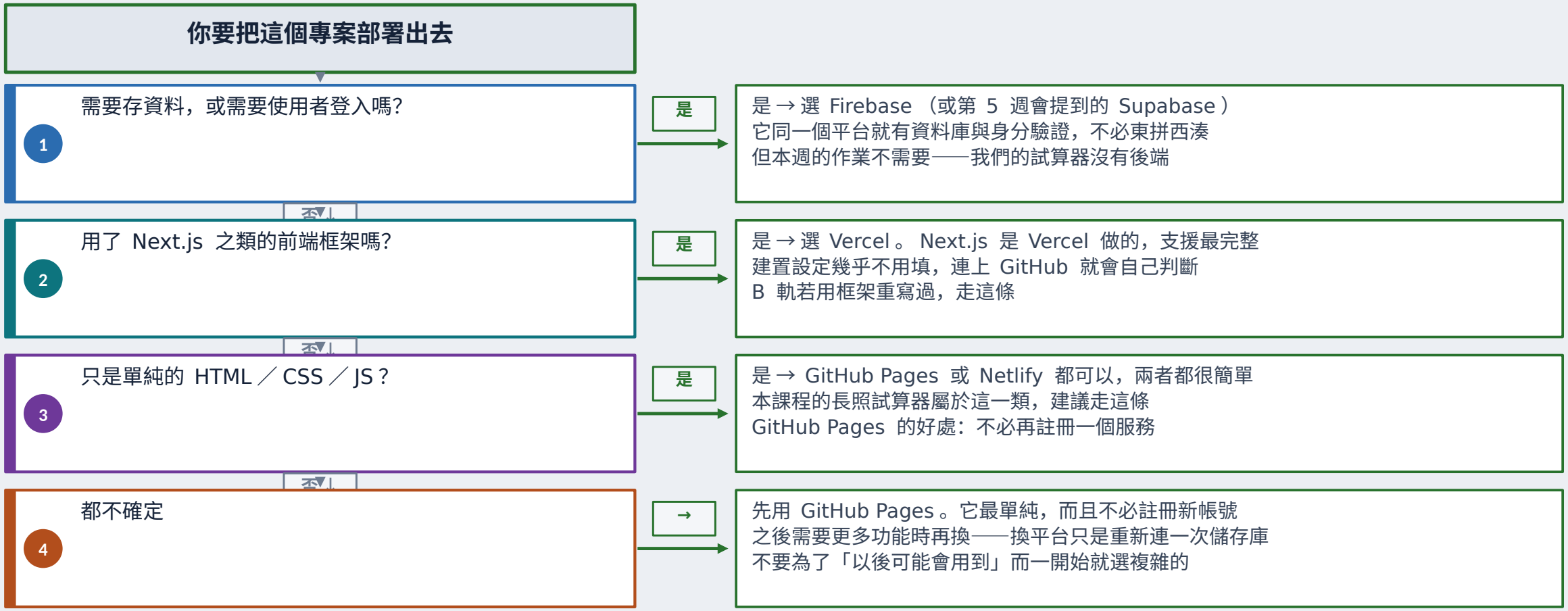
注意

要用指令列（CLI）操作，
多一個步驟；設定較細

※ 這一頁會過時

各平台的方案與額度變動頻繁，本表整理於製作當期。
開課前請重新確認一次免費額度與使用條款——這一頁是獨立的，更新它不影響其他頁面。

選型決策樹：三個問題就決定了



最後一項的原則值得記住：選最單純、能滿足現在需求的那一個。這門課的專案六週都不會需要資料庫，所以最簡單的選項就是最好的選項——複雜度要等到真的需要時再引入。

部署後一定要做的六個檢查

#	檢查什麼	怎麼做	常見問題
1	用別人的裝置開一次	把網址傳給同學或家人，請他們點開	只在自己電腦上開過，等於沒驗證
2	每一個功能都點一次	不是只點你最熟的那條路。 21 種組合再跑一遍	部署後的環境跟本機不完全一樣
3	重新整理看看	按 F5 或重新載入	有些問題只在重新載入時出現
4	開發者工具 Console	在部署後的網址上按 F12 ，看有沒有紅字	本機沒事、線上有事，多半是檔案路徑問題
5	手機實機開一次	用自己的手機點那個網址	模擬器不等於真機。字型、捲動可能不一樣
6	儲存庫全文搜尋金鑰	在 GitHub 儲存庫頁面搜尋 key 、 token 、 password	搜得到就是外流了。立刻到服務商後台撤銷

第 1 項與第 6 項是本週驗收條件。第 6 項尤其重要——期末專題的評分規準裡，發現金鑰外流該項直接零分，所以現在就要養成部署完就搜一次的習慣。

三個常見的部署失敗與解法

症狀	最常見的原因	怎麼確認	怎麼修
本機好好的，部署後一片空白	建置輸出資料夾設錯（publish directory）	F12 → Console 看紅字；Network 分頁看哪個檔案 404	Vite 專案通常是 dist，Create React App 是 build。純 HTML 專案通常留空或填 /
圖片或檔案讀不到	路徑寫成絕對路徑（C:\Users\...）	Console 會出現 404，並顯示它去哪裡找了	一律用相對路徑，或把靜態檔放進 public/
改了推上去，網址沒有更新	① 還沒 push ② 部署還在跑 ③ 瀏覽器快取	到平台後台看部署紀錄（Deployments）	先 Ctrl + F5 強制重新整理。還是舊的就看部署紀錄有沒有失敗

三個裡面第一個最常見，而且解法完全一樣：去 Console 看它到底在找哪一個檔案、為什麼找不到。這就是 VC2 學的東西在部署場景的應用——證據永遠在開發者工具裡。

靜態網站 vs 需要後端：這一頁決定你的部署有多麻煩

比較項目	靜態網站（本課程）	需要後端
是什麼	就是 HTML ／ CSS ／ JS 三種檔案，瀏覽器直接讀	還需要一台伺服器在後面跑程式、存資料
本課程的試算器	屬於這一類。所有計算都在瀏覽器裡完成	—（第 5 週加上雲端紀錄之後才會碰到）
部署難度	極簡。丟上去就好，設定幾乎不用填	要設環境變數、要選執行環境、可能要付費
費用	免費，而且沒有用量上限的問題	多半有免費額度，超過要付費
怎麼判斷自己是哪一種	關掉網路還能用嗎？能 → 靜態	需要「別人也看得到我存的東西」→ 需要後端
金鑰放哪裡	靜態網站藏不住任何金鑰——前端的東西按 F12 都看得到	金鑰放在後端環境變數，前端呼叫自己的後端
這門課的建議	六週都停在靜態。作業與期末專題都做得到	想做的話 VC5 會講清楚界線與代價

> 倒數第二列很重要，第 5 週會整整講一頁：只要是「送到瀏覽器」的東西，使用者按 F12 就看得到，包括你以為藏起來的金鑰。這是靜態網站的根本限制，不是設定問題。

PART 04

動手做： Step 1–6

Hands-on

請打開你上週完成的 v0.3 專案（含 test.html 的那一份）。
Step 4 是本週的重點——故意讓 AI 改壞，再退回來。不要跳過。

Step 1　確認電腦上有沒有 Git，沒有就裝一個

你現在要做什麼　▶　先開終端機，輸入一行指令看看

① 開終端機

Windows：搜尋「cmd」或「PowerShell」
macOS：用 Spotlight　搜尋「終端機」

② 輸入這一行

```
git --version
```

③ 沒有的話去裝（只做一次）

Windows：搜尋「Git for Windows」下載安裝，
　　安裝選項全部用預設值，一路下一步即可
macOS：在終端機輸入　`xcode-select --install`

你應該看到什麼

畫面出現類似：
　git version 2.43.0
版本號不必跟這裡一樣，
有數字就代表裝好了。
裝好之後跳到下一頁設定名字。

沒看到的話，先檢查這幾件事

① 顯示「不是內部或外部命令」
　→ 還沒裝，或裝完沒重開終端機
　→ 關掉終端機再開一次通常就好
② 完全不想碰指令
　→ 看第 28 頁的三條替代路線

Git 只需要裝一次，之後所有專案都能用。安裝過程沒有任何需要判斷的選項——全部按預設值即可，不必看懂那些英文。

Step 1-2　設定你的名字與信箱（只做一次）

你現在要做什麼　▶　在終端機輸入這兩行，把引號裡的內容換成你自己的

兩行設定

```
git config --global user.name " 你的名字 "
git config --global user.email " 你的信箱 "

例:
git config --global user.name "Nan-Chen Hsieh"
git config --global user.email "nchsieh@example.com"
```

確認有設好

```
git config --global user.name
→ 會印出你剛才設的名字
```

你應該看到什麼

輸入後沒有任何訊息，
這是正常的—— Git 成功時多半不說話。
用第二個區塊的指令確認一次，
會印出你的名字就對了。

沒看到的話，先檢查這幾件事

① 引號打成全形「」
　→ 要用英文半形的雙引號 ""
② 不確定有沒有設成功
　→ 跑一次 git config --global user.name
③ 名字可以用中文嗎 → 可以

為什麼要設：每一次 commit 都會記錄「誰改的」。不設的話 Git 在第一次 commit 時會直接擋下來要求你設——現在先弄好，等一下就不會卡住。

不想用指令？三條替代路線（A 軌建議看這頁）

路線	怎麼做	優缺點
① GitHub Desktop（圖形介面）	下載安裝 GitHub Desktop，登入 GitHub 帳號。 之後 commit、push、revert 全部用點的	最推薦給不想碰指令的人。 功能完整，本週要做的每一件事它都做得到
② 瀏覽器工具內建的 GitHub 連結	Lovable、Bolt.new 等多半有「Connect to GitHub」 或「Export to GitHub」按鈕，點一下就連好	最省事，但版本操作（revert） 通常要回到 GitHub 網站或 Desktop 才能做
③ GitHub 網頁直接上傳	在 GitHub 儲存庫頁面點 Add file → Upload files， 把檔案拖進去	只適合第一次。之後每次都手動上傳很痛苦， 而且沒有 commit 訊息的紀律
本課程的建議	A 軌走①（GitHub Desktop），B 軌走指令。	兩者的概念完全一樣（add → commit → push）， 只是一個用點的、一個用打的。講義兩種都會標示

選哪一條都可以，但請至少完成一次「推上 GitHub」。理由是：GitHub 是跨工具的通用格式，而部署平台要從那裡抓程式碼。綁在單一平台的成果，換一個工具就帶不走了。

Step 2　在 GitHub 上建立一個空的儲存庫

你現在要做什麼　▶　先到 github.com 建立一個空的儲存庫（還不要放任何檔案）

網頁上的操作

1. 登入 github.com
2. 右上角 ＋ → New repository
3. Repository name： ltc-calculator
4. Public （課堂專案建議選這個，見第 15 頁）
5. 不要勾 Add a README file
   （勾了會產生一個 commit，第一次推送容易衝突）
6. 按 Create repository

你應該看到什麼

建立完成後會出現一個
「… or push an existing repository」的區塊，
裡面有兩行指令，長得像：

```
git remote add origin https://github.com/...
git push -u origin main
```

這一頁先不要關，下一步會用到。

沒看到的話，先檢查這幾件事

① 名稱被佔用
→ 換一個，例如 ltc-calculator-2026
② 不小心勾了 Add a README
→ 最省事：把這個儲存庫刪掉重建一個空的
（不要用 --force，那會蓋掉雲端那一份）
③ 找不到那兩行指令
→ 重新整理儲存庫首頁就會出現

為什麼要建立「空的」儲存庫：你電腦裡已經有專案了，等一下要把它推上去。如果 GitHub 這邊也先產生了檔案，兩邊的歷史對不起來，第一次推送就會失敗——這是新手最常卡住的地方。

Step 2-2　第一次推送：六個指令，逐一拆解

你現在要做什麼　▶　在你的專案資料夾裡開終端機，依序輸入這六行

六個指令（只做一次）

```
git init
git add .
git commit -m "初版：長照給付試算器，含 29 項測試"
git branch -M main
git remote add origin https://github.com/你的帳號/ltc-calculator.git
git push -u origin main
```

每一行在做什麼

① `init`　在這個資料夾開始使用 Git（產生 `.git` 資料夾）
② `add .`　把所有檔案挑進暫存區（`.` 代表全部）
③ `commit`　拍下第一張快照，並寫一句說明
④ `branch`　把目前這條線改名為 `main`（Git 舊版預設叫 `master`，不改的話第 ⑥ 行會說「`main` 不存在」）
⑤ `remote`　告訴 Git「雲端那一份在這個網址」
⑥ `push`　把本機的 `commit` 上傳上去

你應該看到什麼

每一行執行完都會有一些訊息，
沒有出現紅色的 error 就是成功。

第 ⑥ 行跑完之後，
回到 GitHub 儲存庫頁面重新整理——
你的檔案應該全部出現在那裡了。

沒看到的話，先檢查這幾件事

① 「怎麼在資料夾裡開終端機」
→ Windows：資料夾空白處右鍵，找「在終端機中開啟」
② push 時要求登入
→ 照畫面指示用瀏覽器授權即可
③ 說 main 不存在 → 漏了第 ④ 行，補跑一次

這六行只在「第一次」做。之後每次改完只要三行（下一頁）。動手前請先確認 .gitignore 已經放好——第 16 頁講過，已經 commit 過的檔案就算之後排除也還留在歷史裡。

第一次推送前的四項確認（做完再按 Enter）

#	確認什麼	怎麼做	沒做的後果
1	.gitignore 已經放好	專案資料夾裡有一個 .gitignore 檔， 內容至少包含 .env 與 node_modules/	金鑰或上萬個檔案會被一起推上去， 而且刪不掉（歷史留著）
2	沒有任何真實資料	打開專案資料夾逐一看過檔名， 確認沒有個案清單、名冊之類的東西	本課程紅線。推上公開儲存庫等於公開
3	沒有寫死的金鑰	在編輯器裡全文搜尋 key 、 token 、 password	幾分鐘內就會被自動掃走
4	現在的狀態是能跑的	打開 index.html 與 test.html 各確認一次	第一個 commit 就是壞的，回去也沒用

前三項是不可逆的。推上去之後才發現，唯一有效的處置是「撤銷那把金鑰並重新產生」——刪 commit 救不回來，因為 GitHub 的歷史與各種鏡像早就存下來了。花三分鐘檢查，比事後補救划算太多。

推上去之後： GitHub 頁面上怎麼讀你的紀錄

點進某一次 commit 會看到這樣的畫面

```
修正低收入戶自負額（0.05 → 0）
  nchsieh committed 2 hours ago

  calculator.js

  @@ -30,7 +30,7 @@

   const IDENTITY_RATES = {
     general: { name: '一般戶', rate: 0.16 },
     midlow:  { name: '中低收入戶', rate: 0.05 },
-    low:     { name: '低收入戶', rate: 0.05 },
+    low:     { name: '低收入戶', rate: 0 },
   };
```

（– 開頭是刪掉的，畫面上是紅底）
（＋ 開頭是新增的，畫面上是綠底）

1 最上面是你的訊息

這就是為什麼訊息要寫清楚
一整排 update 的話，這裡完全看不出改了什麼

2 只顯示「改動的那幾行」

不是整個檔案，只有變的部分
上下幾行是給你看脈絡用的

3 – 紅色是刪掉的

原本 rate: 0.05
一眼就看得出改前是什麼

4 ＋ 綠色是新增的

改成 rate: 0
一次改動就只有這一行——這就是好的 commit

這個畫面是「可交接」的核心價值：接手的人不必問你，自己點開歷史就知道每一次改了什麼、為什麼改。而它成立的前提有兩個—— commit 訊息寫得清楚，而且一次只改一件事。

Step 3　之後每一次：三行就好

你現在要做什麼　▶　從現在開始，每完成一個可運行的改動就做這三行

三行（每次都一樣）

```
git add .
git commit -m "修正低收入戶自負額（0.05 → 0）"
git push
```

第二行的引號裡換成「你剛剛改了什麼」

GitHub Desktop 使用者的對應操作

① 左側會列出改過的檔案　＝　add
② 左下角填 Summary 後按 Commit　＝　commit
③ 上方按 Push origin　＝　push

你應該看到什麼

GitHub 儲存庫頁面上，
提交紀錄（Commits）會多一筆。
點進去可以看到「哪幾行被改了」——
新增的是綠色、刪除的是紅色。
本週驗收條件：至少 5 個 commit。

沒看到的話，先檢查這幾件事

① commit 說 nothing to commit
→ 沒有檔案被改過，或忘記存檔
② push 被拒絕（rejected）
→ GitHub 上有你本機沒有的東西。
先 git pull 再 git push
③ 訊息想不出來 → 就寫你剛剛改了什麼

三行一組，養成肌肉記憶。實務上你 95% 的時間就只會用到這三行——其他指令等到需要時再查第 47–48 頁的速查卡。

什麼時候該做那三行

三個一定要 commit 的時機

- ① 每修好一個 bug—— 這樣出事時可以精準退掉某一次修正
- ② 每加完一個功能——功能是自然的分界點
- ③ 要 AI 做大幅修改「之前」——這是最重要的一個

為什麼第 ③ 個最重要

- 它就是你的退路。沒有先 commit，等一下的 revert 就沒有地方可以退
- Step 4 會完整示範這個順序：先 commit → 讓 AI 改壞 → revert 回來
- 養成習慣之後，你會發現自己敢讓 AI 做更大膽的嘗試

一個好用的循環

- commit → 讓 AI 改一件事 → 自己確認 → commit → 讓 AI 改下一件事
- 這樣每一次 AI 的修改都被單獨記錄，出事時可以只退掉那一次
- 對照 VC2 學的「一次只修一個問題」——是同一個原則的兩個面向

一個實用的自我檢查：如果現在電腦突然壞掉，你最多會損失多少工作？超過一小時，代表你 commit 得太少了。

Step 4　本週重點：故意讓 AI 改壞

你現在要做什麼　▶　先確認現在是乾淨可用的狀態，並且已經 commit 過

① 先確認退路存在

```
git status     → 應該顯示 nothing to commit
git log --oneline   → 應該看得到你的 commit 清單
```

如果 status 顯示有未提交的改動，先做完三行再繼續

② 然後故意下一個很糟的指令

請把這個試算器改成「支援多國語言」，
並且重構整個 calculator.js 讓它更有彈性。

這是一個刻意寫得很糟的指令：範圍不明、
沒有限制檔案、沒有驗收條件。

你應該看到什麼

AI 會做出大幅度的改動——
可能拆出好幾個新檔案、
改寫計算邏輯、換掉畫面結構。

然後打開 test.html：
很可能有一堆變紅，甚至整個跑不起來。
這正是我們要的結果。

沒看到的話，先檢查這幾件事

① 它改得還不錯，沒有壞掉
　→ 再加碼：「請一併改用某某框架重寫」
② 你捨不得毀掉現在的版本
　→ 就是因為捨不得才要練。
　　你已經 commit 過了，一定回得去
③ 它拒絕做 → 換一個更大的改動要求

為什麼要刻意做這件事：多數人是在真的出事、心慌意亂的時候第一次用 revert，那時候最容易誤用（例如用了 reset 把東西弄丟）。在可控的情況下練一次，你才會在真的需要時用得出來。

Step 4-2　先找出要退掉的是哪一次

你現在要做什麼　▶　在終端機輸入 git log --oneline，看清楚再動手

輸入

```
git log --oneline
```

你會看到類似這樣的清單

```
a1b2c3d  支援多國語言與重構          ← 就是這一個
e4f5g6h  加上 test.html，29 項測試
i7j8k9l  未選身分別時顯示提示訊息
m0n1o2p  修正第 8 級上限被壓回第 7 級
q3r4s5t  初版：可以算出給付金額
```

左邊那串七個字母就是編號，等一下要用

你應該看到什麼

最上面那一行就是最新的一次，
也就是 AI 剛才改壞的那一次。
把它左邊的七個字母抄下來。

注意：訊息寫得好，這裡就一目了然——
這就是第 9 頁的價值。

沒看到的話，先檢查這幾件事

① 清單很長看不完
→ 按 q 離開；或用 git log --oneline -5 只看最近五筆

② 訊息都是 update 看不出哪一個
→ 這就是為什麼訊息要寫清楚。這次先用時間判斷，下次改進

git log 是安全的——它只顯示，不修改任何東西。動手退回之前先看清楚，比退錯了再退回來省事得多。

Step 4-3　執行 revert，退回乾淨的狀態

你現在要做什麼　▶　用剛才抄下來的編號，執行 revert 並推上去

兩行指令

```
git revert a1b2c3d
git push
```

編號換成你自己抄下來的那七個字母

執行 revert 之後會跳出一個編輯畫面

它要你確認這次 revert 的說明文字。
直接存檔離開即可：

```
Vim（畫面下方有 -- INSERT --  或看起來卡住）：
  按 Esc → 打 :wq → 按 Enter
nano（畫面最下面有一排 ^X ^O 之類）：
  按 Ctrl + O → Enter → Ctrl + X
```

你應該看到什麼

git log 會多出一個新的 commit：
　Revert " 支援多國語言與重構 "
原本那一次還留在歷史裡，
但它造成的修改被抵銷掉了。

打開 test.html → 29 項應該全部回到綠色。

沒看到的話，先檢查這幾件事

① 卡在看不懂的編輯畫面出不來
　→ Vim： Esc → :wq → Enter
　→ nano： Ctrl+O → Enter → Ctrl+X
② 出現 conflict 字樣
　→ 中間隔了其他 commit。把訊息
　　整段貼給 AI 問怎麼處理
③ 想確認退對了 → 直接跑 test.html

注意 revert 的做法：它不是「刪掉」那次 commit，而是「再做一次相反的修改」。所以歷史完整保留，別人的紀錄也不會對不上——這就是為什麼推上 GitHub 之後一律用 revert，不要用 reset。

三種退路，什麼時候用哪一個（動手前先看這張）

revert

什麼情況用它

已經推上 GitHub 了

指令

```
git revert < 編號 >
```

會發生什麼

新增一個「做相反修改」的 commit。
歷史完整保留，最安全

reset

什麼情況用它

還在自己電腦裡，
想把剛才的實驗整段丟掉

指令

```
git reset --hard < 編號 >
```

會發生什麼

把指標搬回去，之後的 commit
從紀錄上消失。推過就不要用

checkout

什麼情況用它

只是想看看
上週那版長什麼樣

指令

```
git checkout < 編號 >
```

會發生什麼

跳過去看，什麼都沒改。
看完用 git checkout main 跳回

搞不清楚？先不要動

先跑 git status 與 git log --oneline—— 這兩個指令永遠是安全的（只顯示，不修改）。
把輸出整段貼給 AI，問「我現在在什麼狀態？該怎麼辦？」看清楚再動手。

底下那條紅色的是最實用的一項： Git 出問題時最糟的做法是「亂試指令看看會怎樣」。 git status 與 git log 這兩個指令永遠是安全的（它們只顯示，不修改），先看清楚再動手。

Step 4-4　記錄這一次 revert （本週交付物之一）

你現在要做什麼　▶　把剛才的過程記下來，四個欄位

要記錄的四件事

① 改壞前：畫面正常， test.html 29/29　全綠
　（附一張截圖）

② 我下了什麼指令給 AI：「請改成支援多國語言…」

③ 改壞後： test.html　顯示 X　項失敗（附截圖）

④ 我用了什麼指令退回： git revert a1b2c3d → git push
　退回後 test.html　恢復 29/29 （附截圖）

你應該看到什麼

一份含三張截圖的簡短紀錄。

重點不在格式，
而在你真的走過一次完整的
「commit → 改壞 → revert → 驗證」循環。

這一份是本週的驗收條件之一。

沒看到的話，先檢查這幾件事

① 忘記在改壞前截圖
　→ 沒關係，再做一次。
　　這次記得先截圖再改壞
② 不知道怎麼截圖
　→ Windows： Win + Shift + S；
　　macOS： Shift + Command + 4

三張截圖分別對應「有退路」「出事了」「退回來了」。這個循環走過一次之後，你對 AI 做大幅修改的態度會完全不同——不是不敢讓它改，而是知道改壞了三十秒就能回來。

Step 5　部署成一個公開網址

你現在要做什麼　▶　選一個平台，連上你的 GitHub 儲存庫（以 GitHub Pages 為例）

GitHub Pages （最單純，不必註冊新服務）

1. 到你的儲存庫頁面 → 上方 Settings
2. 左側選 Pages
3. Source 選 Deploy from a branch
4. Branch 選 main、資料夾選 / (root)
5. 按 Save，等 1–2 分鐘

Netlify / Vercel （走「連 GitHub」路線）

1. 用 GitHub 帳號登入該平台
2. Add new site → Import an existing project
3. 選你的儲存庫
4. 純 HTML 專案：build command 留空、publish directory 留空或填 /
5. 按 Deploy

你應該看到什麼

過一兩分鐘後會出現一個網址，
類似：
https:// 你的帳號 .github.io/ltc-calculator/

點進去，你的試算器應該正常顯示。
把這個網址複製起來——
它是本週最主要的交付物。

沒看到的話，先檢查這幾件事

① 顯示 404
→ 等久一點（第一次可能要 3–5 分鐘）；或確認 index.html 在儲存庫根目錄
② 一片空白
→ F12 看 Console。多半是路徑問題，見第 23 頁的三個常見失敗
③ 樣式跑掉 → 檢查檔案是否都推上去了

設定完成之後就是自動的了：以後你只要 git push，網址上的內容會在一兩分鐘內自動更新，不必再回來做任何事。這就是把部署接到 GitHub 的價值。

Step 6　部署後六個檢查，一項一項做

你現在要做什麼　▶　打開第 22 頁的檢查清單，六項逐一完成

六項檢查

1. 用別人的裝置開一次（傳給同學或家人）
2. 每一個功能都點一次（21 種組合再跑一遍）
3. 重新整理看看
4. 在部署後的網址上按 F12，看 Console 有沒有紅字
5. 用自己的手機實機開一次
6. 到 GitHub 儲存庫搜尋 key、token、password

第 6 項怎麼搜

在儲存庫頁面按鍵盤的 /　（或點上方搜尋框）
分別輸入 key、token、password、secret
搜得到就是外流了 —— 立刻到服務商後台撤銷

你應該看到什麼

六項全部打勾。

第 1 項與第 6 項是本週驗收條件：
助教會用自己的手機開你的網址，
並實際搜一次你的儲存庫。

搜到金鑰即該項未通過。

沒看到的話，先檢查這幾件事

① 手機上開起來破版
→ 回到 VC2 第 38 頁的做法處理
② Console 有紅字但畫面看起來正常
→ 還是要處理。它代表有東西
載入失敗，只是剛好不影響主功能
③ 搜到金鑰 → 先撤銷，再處理程式碼

這個專案不需要任何金鑰，所以第 6 項應該搜不到東西。但這個習慣要現在養成——第 5 週會實際用到金鑰，到時候這一步就是真的在把關了。

做完這一週，你的專案資料夾長什麼樣

名稱	是什麼	要注意什麼
index.html	畫面與互動	推上去的主角。必須在儲存庫最外層，否則部署會 404
calculator.js	計算邏輯	VC3 拆出來的。改它會讓測試變紅，這是好事
test.html	29 項測試	每次改完重跑一次。推上 GitHub 也看得到
README.md	專案說明	VC6 才會正式寫。但現在可以先把網址貼進去
.gitignore	排除清單	這個檔案本身「要」推上去——它是專案設定的一部分
.git ／（隱藏）	Git 的完整歷史都在這裡	千萬不要刪掉或搬動它。刪了就沒有歷史了。 它預設是隱藏的，看不到是正常的

最後一列值得特別記住： .git 這個隱藏資料夾就是「版本控制」本身。複製專案給別人時如果只複製看得到的檔案，歷史就不會跟著走——這也是為什麼要用 git clone ，而不是複製資料夾。

網址拿到之後，三件該做的事

① 把網址寫進 README.md

- 在 README 最上面加一行：「線上版本：〔你的網址〕」
- 別人打開你的儲存庫，第一眼就看得到成品
- VC6 會把 README 寫完整，現在先放這一行就好

② 找三個人實際用一次

- 不要說明、不要在旁邊教，就把網址傳給他們
- 看他們會不會用錯、會不會誤解那些數字的意思
- 他們卡住的地方就是你的問題——記下來，VC6 的同儕審查會用到

③ 存進你的期末專題交付清單

- 期末五項交付物裡，② 公開網址與 ③ GitHub 儲存庫這兩項現在就完成了
- 加上 VC1 的規格書、VC3 的驗證報告，你已經有四項
- 剩下 ⑤ 交接文件是 VC6 的內容——期末專題其實已經完成八成

第 ② 項最有價值，也最容易被跳過。你太熟悉自己的專案了，熟到看不見它哪裡令人困惑——而使用者第一次打開時，正是那種狀態。

卡住了？照症狀查（一） Git 與 GitHub

症狀	可能原因	怎麼確認	怎麼修
git 不是內部或外部命令	沒裝，或裝完沒重開終端機	git --version	關掉終端機再開一次。 還是不行就重裝 Git for Windows
push 被拒絕 **（rejected）**	GitHub 上有你本機沒有的東西	git status git log --oneline	先 git pull，再 git push。 出現衝突就把訊息貼給 AI 問
commit 時說 **nothing to commit**	沒有檔案被改過， 或忘記存檔	git status 看有沒有 列出檔案	先在編輯器存檔，再 git add .
卡在看不懂的 **編輯畫面出不來**	那是 Git 開起來的文字編輯器 （Vim 或 nano）	畫面下方有 -- INSERT -- 是 Vim； 有一排 ^X ^O 是 nano	Vim： Esc → :wq → Enter nano： Ctrl+O → Enter → Ctrl+X
push 一直要求登入	GitHub 已不接受密碼登入	看它跳出的是不是 瀏覽器授權視窗	照畫面指示用瀏覽器授權； 或改用 GitHub Desktop
不小心把金鑰 **推上去了**	.gitignore 沒設或設太晚	在儲存庫搜尋 key、token、password	第一件事：到服務商後台撤銷那把金鑰 並重新產生。刪 commit 救不回來

> 通則： Git 出問題時， git status 與 git log --oneline 這兩個指令永遠是安全的——它們只顯示，不修改任何東西。先看清楚現在的狀態，把輸出整段貼給 AI，再動手。

卡住了？照症狀查（二）部署

症狀	可能原因	怎麼修
網址開起來 404	① 還在部署中 ② index.html 不在根目錄 ③ 設定還沒生效	先等 3–5 分鐘。仍是 404 就確認 index.html 在儲存庫最外層，且 Pages 設定的分支是 main
一片空白	建置輸出資料夾設錯， 或檔案路徑寫成絕對路徑	F12 → Console 看紅字、 Network 看哪個檔 404 。 純 HTML 專案 publish directory 留空或填 /
樣式跑掉、圖片不見	檔案沒推上去，或路徑大小寫不符	到 GitHub 網頁確認檔案真的在那裡。 Linux 伺服器分大小寫， Windows 本機不分
改了推上去沒更新	① 忘記 push ② 部署失敗 ③ 瀏覽器快取	先 Ctrl + F5 強制重新整理。 再到平台後台看 Deployments 有沒有失敗紀錄
手機開起來破版	沒有 viewport 設定或用了固定寬度	回到 VC2 第 38 頁的做法。 模擬器過了不代表真機過，兩個都要看

部署問題的共同解法：在「部署後的那個網址」上按 F12 ，不是在本機檔案上。線上與本機的環境不一樣，證據要去線上撈——這是 VC2 學的方法在新場景的應用。

常見的四個誤解（做完之後回頭看一次）

誤解	為什麼是錯的	正確的做法
① commit 就等於備份到雲端	commit 只存在你的電腦裡。 電腦壞掉一樣沒了	commit 之後要 push 才會上傳。 三行指令的第三行就是它
② 改壞了就用 reset 退回去	推上 GitHub 之後用 reset， 會讓紀錄對不上	推過就一律用 revert。 reset 只留給「還沒推上去的本機實驗」
③ .gitignore 之後再補就好	已經 commit 過的檔案， 之後排除也還留在歷史裡	第一次 commit 之前就要設好。 這是金鑰外流最常見的原因
④ 在自己電腦上開得起來就算好了	本機與線上的環境不一樣： 路徑大小寫、建置設定都可能不同	部署後六個檢查一項不能少， 特別是「用別人的裝置開一次」

> 第 ③ 個誤解的代價最高，而且不可逆。金鑰一旦進了 Git 歷史，唯一有效的處置是到服務商後台撤銷並重新產生——這件事第 5 週會再講一次，因為那時候你真的會用到金鑰。

指令速查卡（一）每天都會用到的

情境	指令	說明
每次改完（最常用）	git add . git commit -m " 說明 " git push	三行一組。每完成一個可運行的改動就跑一次。 實務上 95% 的時間只會用到這一組
搞不清楚現在什麼狀態	git status git log --oneline	這兩個永遠安全，只顯示不修改。 出狀況時先跑這兩個，把輸出貼給 AI 問
第一次設定（只做一次）	git init git branch -M main git remote add origin < 網址 > git push -u origin main	init 開始在這個資料夾使用 Git； branch -M main 把分支改名（舊版預設是 master）； remote 綁定雲端位置

> GitHub Desktop 使用者可以對照： add ＝勾選左側的檔案、 commit ＝填 Summary 後按 Commit 、 push ＝按 Push origin 。概念完全一樣，只是一個用點的、一個用打的。

指令速查卡（二）偶爾才會用到的

情境	指令	說明
退回某一次修改	git revert < 編號 > git push	已經推上 GitHub 就用這個。 新增一個「做相反修改」的 commit，歷史保留
丟掉還沒推的本機改動	git reset --hard < 編號 >	只用在「還沒 push」的情況。 會真的把之後的 commit 丟掉
看看舊版長什麼樣	git checkout < 編號 > git checkout main	跳過去看，什麼都不會改。 看完記得用第二行跳回來
把專案抓到另一台電腦	git clone < 網址 >	完整歷史一起下來
卡在編輯畫面出不來	Vim： Esc → :wq → Enter nano： Ctrl+O → Enter → Ctrl+X	不是 Git 指令，是編輯器的。 revert 時會遇到

這一頁不必背，就是給你查的。建議把這兩頁印出來貼在螢幕旁邊——尤其是「搞不清楚狀態時先跑 status 與 log」這一條，它會在你最慌的時候救你。

本週自我檢核（答不出來請回去看標註的頁數）

Q 1 **commit 跟「另存新檔」差在哪？做完 commit 東西上雲端了嗎？**

commit 是快照不是覆蓋；而且只在本機，要 push 才上傳 → 第 6 頁與第 46 頁

Q 2 **revert 與 reset 差在哪？已經推上 GitHub 的話該用哪一個？**

revert 新增相反的 commit、歷史保留；推過就一律用 revert → 第 10 頁與第 38 頁

Q 3 **.gitignore 為什麼一定要在第一次 commit 之前設好？**

已 commit 過的檔案即使之後排除也還留在歷史裡 → 第 16 頁

Q 4 **部署後的網址開起來一片空白，你第一個動作是什麼？**

在那個網址上按 F12 看 Console 與 Network → 第 23 頁與第 45 頁

第二題是本週最重要的一題。你已經實際做過一次 revert （Step 4），所以應該答得出來——如果只是看過沒做過，請回去把 Step 4 補做完，那一步不能只用讀的。

本週交付與驗收條件

交付物	驗收條件（全部達成才算通過）	怎麼查
① **公開網址**	別人（含手機）打得開、跑得動， 21 種組合都正確	助教用自己的手機開一次
② **GitHub 儲存庫**	至少 5 個 commit ，且訊息看得出改了什麼 （不得全是 update ／ fix ）	看提交紀錄
③ **revert 紀錄**	含三張截圖：改壞前（測試全綠）／改壞後（測試變紅）／ 退回後（恢復全綠），並寫出用了什麼指令	文件檢查
④ **部署後六個檢查**	六項全數勾選	檢核表
⑤ **無金鑰外流**	儲存庫全文搜尋 key 、 token 、 password 都搜不到	助教實際搜一次。搜到即未通過

> 第 ③ 項是本週的核心。它不是為了交作業——走過一次完整的「有退路 → 出事 → 退回來」循環之後，你對 AI 做大幅修改的態度會完全不同。這也是期末專題敢放手做的前提。

第 4 週重點回顧

1. commit 不是存檔，是拍一張可以回去的快照
2. commit 只在本機， push 才會上傳。訊息要讓三個月後的你看得懂
3. 推上 GitHub 之後一律用 revert ， reset 只留給本機實驗
4. .gitignore 要在第一次 commit 之前設好——歷史刪不掉
5. 部署就是把檔案放到一台永遠開著的機器上，對這個專案完全免費
6. 你現在敢讓 AI 做大幅修改了——因為你知道回得去

下一步　下一週：資料、風險與成本——資料能存在哪、金鑰要放哪裡、什麼資料不能貼進 AI 工具，以及這樣做一個月大約要花多少錢。